

Construindo um app WebView do zero

VERSÃO 2.0 · ATUALIZADA

Tutorial passo a passo: do VS Code e do Git até o app publicado na Google Play, com o Android Studio atual, a escolha correta do JDK e o repositório organizado — e como reaproveitar o mesmo molde para outros clientes.

Antes de começar

Este material monta, do zero, um app WebView: um navegador sem barra de endereço, empacotado como app nativo, que abre um sistema web dentro de uma janela com a marca do cliente.

O exemplo do começo ao fim é o app da Sayan, que abre a área do beneficiário em sayan.bemestar.inorte.tech. Mas a intenção é outra: este é um **molde**. Trocando poucas coisas — endereço, identificador, nome, cores e logo — o mesmo projeto vira o app de qualquer cliente que já tenha um sistema web. A Parte 6 trata exatamente disso.

Esta é a **versão 2**. Ela junta o tutorial original com o que aprendemos ao montar o projeto de verdade, no ambiente novo: Android Studio atual, escolha correta do JDK e versionamento com Git dentro do VS Code. O PDF vive no repositório, ao lado do código que ele descreve. Você vai criar uma pasta nova e digitar cada arquivo — não é o repositório de produção da Sayan, é o molde, para você entender cada peça antes de reaproveitá-la.

DICA

Digite o código, não copie e cole. Errar uma chave e ter que achar o erro é metade do aprendizado. São poucos arquivos e nenhum deles é longo. Se algo não bater, compare com o repositório.

Ao final você terá: o app rodando no emulador, um repositório Git limpo, um **.aab** assinado, o caminho completo até a Play Store e um projeto pronto para virar o próximo app.

O que mudou em relação à versão 1

TEMA	NA VERSÃO 1	NESTA VERSÃO
Android Studio	Instalação com o assistente padrão.	Ambiente atual (Otter 2025.2.1 ou mais novo), SDK 36, Device Manager, Gradle JDK e avisos de atualização explicados.
JDK	"Sempre o 17".	Regra baseada no seu projeto: 17 ou 21, nunca o 25. Como conferir, como escolher e como o terminal usa o JDK.
Git	Só as linhas de segurança do .gitignore, no fim.	Parte 3 inteira: .gitignore antes do primeiro add, commit, GitHub, CRLF e como se recuperar de um <code>git add</code> . errado.
Assinatura	Assumia que a chave já existia.	Como criar a chave de upload e ligar o Gradle ao keystore.properties.
Estrutura	Não mostrada.	Árvore de pastas final e o que vai (ou não) para o repositório.
Erros	Dez linhas.	Tabela ampliada com erros de JDK, Git e Android Studio.

Quando este molde serve — e quando não serve

SERVE BEM	NÃO SERVE
O cliente já tem um sistema web que funciona bem no celular e você quer dar a ele presença na loja de apps.	O app precisa funcionar sem internet, usar câmera, GPS, biometria ou qualquer recurso do aparelho.
O sistema é de terceiros e você não controla o código dele.	O produto é o app em si — aí vale investir em nativo ou React Native.
O prazo é curto e o orçamento não comporta refazer telas que já existem.	O destino é a App Store da Apple: a diretriz 4.2 rejeita com frequência apps que são "só um site embrulhado". O Google Play é bem mais tranquilo.

Roteiro deste material

PARTE	O QUE VOCÊ FAZ	TEMPO ESTIMADO
1 · Preparar a máquina	Node, Git, VS Code, Android Studio, SDK e JDK.	1 h (os downloads são longos)
2 · Criar o projeto	Pasta, repositório, Capacitor e os arquivos do app.	40 min
3 · Versionar com Git	Commit, GitHub, clonar em outra máquina, recuperar erros.	20 min
4 · Rodar e dar acabamento	Android Studio, emulador, ícone, splash e push.	40 min
5 · Publicar	Assinatura, .aab e Play Console.	1 h + revisão do Google
6 · Reaproveitar	Trocar os oito itens e gerar o próximo app.	3 h por app

PARTE 1 — INSTALAR (SÓ UMA VEZ)

Preparar a máquina

1 Instale o Node.js

Baixe em nodejs.org a versão LTS mais nova (o Capacitor atual pede Node 22 ou superior) e instale clicando em Avançar até o fim. Para conferir, abra o PowerShell e digite:

```
node -v  
npm -v
```

POWERSHELL

Tem que aparecer algo como `v22.x.x`. Se aparecer “não é reconhecido”, feche e abra o terminal de novo — a instalação só vale em janelas novas.

2 Instale o Git

Baixe em git-scm.com/download/win e instale com as opções padrão. Depois, apresente-se ao Git — sem isso o primeiro commit falha com “Author identity unknown”:

```
git --version  
git config --global user.name "Seu Nome"  
git config --global user.email "voce@empresa.com"
```

POWERSHELL

3 Instale o VS Code

Baixe em code.visualstudio.com. Na instalação, marque “Adicionar ao menu de contexto do Explorer” — facilita abrir pastas depois. A extensão **Prettier** é opcional, mas deixa o código arrumado sozinho. O VS Code já traz o painel **Source Control** (ícone de ramificação na barra lateral), que vamos usar na Parte 3.

4 Instale o Android Studio

Baixe em developer.android.com/studio. Este material foi escrito para o **Android Studio Otter (2025.2.1)** ou mais novo — é o mínimo que o Capacitor 8 pede; versões antigas dão erros de Gradle que não explicam nada. Na primeira abertura roda um assistente: escolha **Standard** e aceite as licenças. Ele baixa o SDK do Android — são vários gigabytes, deixe rodando.

5 Confirme os componentes do SDK

No Android Studio abra **More Actions** > **SDK Manager** (ou **File** > **Settings** > **Languages & Frameworks** > **Android SDK**) e confira:

ABA	MARQUE
SDK Platforms	Android 16 (API 36) — é o nível que o Capacitor 8 e a Play Store exigem.
SDK Tools	Android SDK Build-Tools, Android SDK Platform-Tools, Android Emulator e Android SDK Command-line Tools.

NOTA

O emulador do Windows depende de **virtualização ligada na BIOS** e do recurso “Plataforma do Hipervisor do Windows”. Se o emulador não abrir ou ficar lento, comece por aí. Ou use seu celular (passo 25).

6 Escolha o JDK (atenção especial)

O JDK é o Java que o Gradle usa para compilar o app. É o ponto que mais quebra build, e a mensagem de erro não ajuda. **A regra:** use um JDK LTS — **17 ou 21** — igual ou mais novo do que o projeto pede, e **nunca o 25**. Algumas versões do Android Studio trazem um JDK embutido mais novo do que o Gradle do projeto entende (já vimos o Java 25). Por isso a escolha tem que ser **explícita**, não o padrão.

JDK	VERSÃO DO CLASS FILE	SITUAÇÃO
17	61	Funciona com Gradle 8.x. Serve para projetos mais antigos, gerados com Capacitor 6 ou anterior.
21	65	Recomendado com o Capacitor 8 (o guia do próprio Capacitor recomenda Java 21).
25	69	Não use. O Gradle 8.x não lê: “Unsupported class file major version 69”.

Como saber o que o seu projeto pede. Depois do `npx cap add android` (passo 17), abra três arquivos:

ARQUIVO	O QUE OLHAR
<code>android/gradle/wrapper/gradle-wrapper.properties</code>	A linha <code>distributionUrl</code> : a versão do Gradle (o Capacitor 8 usa 8.14.x).
<code>android/app/capacitor.build.gradle</code>	O bloco <code>compileOptions</code> : o número em <code>JavaVersion.VERSION_XX</code> é o mínimo de Java.
<code>android/variables.gradle</code>	<code>compileSdkVersion</code> e <code>targetSdkVersion</code> (36 no Capacitor 8) e <code>minSdkVersion</code> (24).

Como instalar e escolher no Android Studio. File > Settings > Build, Execution, Deployment > Build Tools > Gradle. No campo **Gradle JDK**, abra a lista > **Download JDK** > escolha a versão (17 ou 21) > Download. Anote o caminho; normalmente é:

CAMINHO TÍPICO

`C:\Users\<seu-usuario>\.jdk\jbr-21.0.x` # o nome muda conforme versão e fornecedor

O terminal não usa a mesma configuração do Android Studio. O Studio obedece o campo Gradle JDK; o `.\gradlew` no terminal obedece a variável `JAVA_HOME`. Em cada janela do terminal, antes de gerar o pacote:

POWERSHELL (VALE SÓ NESTA JANELA)

```
$env:JAVA_HOME = "C:\Users\<seu-usuario>\.jdk\jbr-21.0.x"
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
java -version
.\gradlew --version # a linha "JVM:" é a prova de qual JDK o Gradle está usando
```

ATENÇÃO

Para descobrir qual JDK o Android Studio está usando, olhe `android/.gradle/config.properties`. E **não** escreva `org.gradle.java.home` no `gradle.properties` do repositório: é um caminho da sua máquina e quebra o build de todo mundo que clonar. Anote no README do repositório as versões de Android Studio, JDK, Gradle e Capacitor que funcionaram.

Montando o app

7 Crie a pasta e abra no VS Code

Crie a pasta `meu-app-webview` num caminho **curto**, como `C:\meu-app-webview`. Evite Área de Trabalho e pastas sincronizadas com OneDrive: o Gradle cria milhares de arquivos e o caminho longo do Windows e a sincronização atrapalham. No VS Code: File > Open Folder e selecione a pasta.

8 Abra o terminal integrado

Menu Terminal > New Terminal, ou o atalho **Ctrl + `** (Ctrl e a tecla de aspas simples). O terminal abre na parte de baixo, já dentro da pasta do projeto, em PowerShell — é ali que todo comando deste material é digitado. Para interromper um comando travado, use **Ctrl + C** uma vez e espere.

9 Inicie o Git e crie o `.gitignore` — antes de tudo

Este é o passo que evita a dor de cabeça da Parte 3. Comece o repositório e, **antes de instalar qualquer coisa**, crie o arquivo `.gitignore` na raiz (botão direito na área de arquivos > New File):

POWERSHELL

```
git init
git branch -M main
```

`.gitignore`

```
# dependências: nunca vão para o Git
node_modules/

# build e caches do Android
android/build/
android/app/build/
android/.gradle/
android/local.properties
android/app/release/

# segredos: NUNCA commitar
keystore.properties
*.jks
*.keystore
.env

# binários gerados
*.apk
*.aab

# editor, sistema e logs
.idea/
.DS_Store
*.log
```

Crie também o `.gitattributes`, que resolve os avisos de finais de linha (explicados no passo 21):

`.gitattributes`

```
* text=auto
gradlew text eol=lf
*.bat text eol=crlf
*.sh text eol=lf
```

10 Inicie o projeto Node

```
npm init -y
```

POWERSHELL

Isso cria o arquivo `package.json`: a “ficha” do projeto, com o nome e a lista de bibliotecas usadas.

11 Instale o Capacitor

O Capacitor transforma conteúdo web em app nativo. Este material foi escrito para o Capacitor 8. Instale as três partes:

```
npm install @capacitor/core @capacitor/android @capacitor/splash-screen
npm install --save-dev @capacitor/cli
```

POWERSHELL

Demora um ou dois minutos. A pasta `node_modules` vai aparecer — nunca mexa nela nem envie para o Git (é para isso que o passo 9 existe). Registre a versão instalada com `npm ls @capacitor/core`.

12 Crie o arquivo de configuração do app

Na raiz do projeto, crie o arquivo com o nome exato `capacitor.config.json`:

```
{
  "appId": "tech.nortel.sayan.beneficios",
  "appName": "Sayan Benefícios",
  "webDir": "www",
  "server": {
    "androidScheme": "https",
    "allowNavigation": [
      "sayan.bemestar.lnorte.tech",
      "*.bemestar.lnorte.tech",
      "*.lnorte.tech"
    ]
  },
  "plugins": {
    "SplashScreen": {
      "launchShowDuration": 1500,
      "launchAutoHide": false,
      "backgroundColor": "#031835",
      "showSpinner": false,
      "androidScaleType": "CENTER_CROP"
    }
  }
}
```

capacitor.config.json

CHAVE	O QUE SIGNIFICA
<code>appId</code>	A identidade do app nas lojas. Depois de publicado, nunca pode mudar nem ser reaproveitado por outro app.
<code>appName</code>	O nome que aparece embaixo do ícone no celular.
<code>webDir</code>	A pasta onde ficam os arquivos web. Vamos criá-la no próximo passo.
<code>allowNavigation</code>	Os domínios que a WebView tem permissão de abrir. Link fora da lista abre no navegador do celular, fora do app.
<code>launchAutoHide</code>	Em false, a splash só some quando o código mandar. É o que dá controle sobre o momento da troca de tela.

ATENÇÃO

Por que o `allowNavigation` importa tanto: no app real, o checkout fica em `sayanbeneficios.com`, que está fora da lista **de propósito**. Assim os dados de pagamento nunca passam pelo app — e foi isso que permitiu declarar na Play Store que o app não coleta dados financeiros. Mudar essa lista muda o que o app precisa declarar.

13 Crie a pasta `www` e o arquivo de ajustes

Crie a pasta `www` e, dentro dela, o arquivo `config.js`. Separar as configurações num arquivo só é um hábito que vale levar para qualquer projeto: quem for trocar a URL amanhã não precisa entender o resto do código.

`www/config.js`

```
// Tudo que muda com frequência fica aqui.
// Depois de editar, sempre rode: npx cap sync
window.APP_CONFIG = {
  // endereço que o app abre depois da splash
  SYSTEM_URL: "https://sayan.bemestar.lnorte.tech/beneficiary/login",

  // deixe "" para desligar o push por enquanto
  ONESIGNAL_APP_ID: ""
};
```

14 Crie a tela do app

Ainda em `www`, crie o `index.html`. São só duas telas: a de carregamento e a de “sem conexão”. Coloque também um `logo.png` dentro de `www` (a logo branca, uns 400 px de largura). Repare que `config.js` é carregado **antes** de `app.js`; se fosse ao contrário, o código tentaria ler uma configuração que ainda não existe.

`www/index.html`

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Sayan Benefícios</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div id="loading" class="tela">
    
    <div class="spinner"></div>
    <p class="msg">Abrindo o sistema...</p>
  </div>

  <div id="offline" class="tela oculto">
    
    <h1>Sem conexão</h1>
    <p class="msg">Verifique sua internet e tente de novo.</p>
    <button id="retry" class="botao">Tentar novamente</button>
  </div>

  <script src="config.js"></script>
  <script src="app.js"></script>
</body>
</html>
```

15 Crie o visual

www/styles.css

```
/* as cores da marca ficam nas duas variáveis abaixo */
:root {
  --brand: #031835; /* navy da marca */
  --accent: #04B16E; /* verde da marca */
}
* { box-sizing: border-box; }
html, body {
  margin: 0; height: 100%;
  background: var(--brand); color: #fff;
  font-family: -apple-system, "Segoe UI", Roboto, sans-serif;
}
.tela {
  height: 100%; display: flex; flex-direction: column;
  align-items: center; justify-content: center;
  padding: 24px; text-align: center;
}
.oculto { display: none; }
.logo { width: 180px; max-width: 60%; margin-bottom: 28px; }
.spinner {
  width: 34px; height: 34px;
  border: 3px solid rgba(255,255,255,0.2);
  border-top-color: var(--accent);
  border-radius: 50%;
  animation: girar 0.9s linear infinite;
}
@keyframes girar { to { transform: rotate(360deg); } }
.msg { color: #b8c8dc; font-size: 15px; margin-top: 18px; }
h1 { font-size: 22px; margin: 0 0 8px; }
.botao {
  margin-top: 22px; padding: 14px 26px; border: 0;
  border-radius: 10px; background: var(--accent);
  color: #fff; font-size: 16px; font-weight: 600;
}
```

16 Crie a lógica

Último arquivo de www: **app.js**, o coração do app, com menos de 50 linhas.

www/app.js

```
(function () {
  var cfg = window.APP_CONFIG || {};
  var jaIniciou = false;

  // troca qual das duas telas está visível
  function mostrar(id) {
    document.getElementById("loading").classList.add("oculto");
    document.getElementById("offline").classList.add("oculto");
    document.getElementById(id).classList.remove("oculto");
  }

  // sai da nossa tela e entra no sistema da lNorte
  function abrirSistema() {
    if (!navigator.onLine) {
      mostrar("offline");
      return;
    }
    window.location.replace(cfg.SYSTEM_URL);
  }

  function iniciar() {
    if (jaIniciou) return;
  }
})
```

```

    jaIniciou = true;

    // esconde a splash nativa
    if (window.Capacitor && Capacitor.Plugins.SplashScreen) {
        Capacitor.Plugins.SplashScreen.hide();
    }
    setTimeout(abrirSistema, 600);
}

// o app nativo avisa quando está pronto
document.addEventListener("deviceready", iniciar, false);

// rede de segurança: se o aviso não vier em 3 s, segue assim mesmo
setTimeout(iniciar, 3000);

document.getElementById("retry").addEventListener("click", abrirSistema);
})();

```

NOTA

O detalhe mais importante do arquivo: `window.location.replace(...)` carrega o sistema da 1Norte. Com um `<iframe>` o site seria bloqueado — quase todo site sério envia o cabeçalho `X-Frame-Options` justamente para não ser embutido em outra página. Aqui é a própria `WebView` nativa carregando o endereço: não existe `iframe` e não existe bloqueio.

17 Gere o projeto Android

```
npx cap add android
```

POWERSHELL

Aparece a pasta **android**, com um projeto Android completo gerado automaticamente — você não escreveu uma linha de Java ou Kotlin. Agora é a hora de conferir os três arquivos do passo 6 e confirmar qual JDK o projeto pede.

18 Sincronize

```
npx cap sync
```

POWERSHELL

O comando que você mais vai esquecer. Ele copia o que está em `www/` e no `capacitor.config.json` para dentro do projeto Android. Mexeu em qualquer um dos dois e não rodou `cap sync`? Nada muda no app. É a causa número um de “editei e não aconteceu nada”.

Como a pasta deve estar agora

ESTRUTURA

```
meu-app-webview/
├── .gitignore                # node_modules, chaves, builds
├── .gitattributes            # finais de linha (LF/CRLF)
├── capacitor.config.json     # identidade e domínios do app
├── package.json / package-lock.json
├── docs/tutorial-app-webview-v2.pdf # este material
├── resources/                # icon.png e splash.png (passo 27)
├── www/                      # o "site" do app
│   ├── app.js config.js index.html logo.png styles.css
├── android/                  # projeto nativo gerado pelo Capacitor
│   ├── app/build.gradle      # applicationId, versionCode, assinatura
│   ├── app/capacitor.build.gradle
│   ├── gradle/wrapper/gradle-wrapper.properties
│   ├── build.gradle settings.gradle gradle.properties
│   ├── variables.gradle      # compileSdk, targetSdk, minSdk
│   ├── gradlew gradlew.bat
│   └── keystore.properties   # Só na sua máquina (passo 31)
└── node_modules/            # fora do Git
```

PARTE 3 — VERSIONAR COM GIT

Repositório limpo desde o primeiro commit

19 Confira o que o Git enxerga

Abra o painel **Source Control** do VS Code (ou rode o comando abaixo). O número no ícone mostra quantos arquivos mudaram. Num projeto saudável são **algumas dezenas** — no nosso caso, 62. Se aparecerem milhares, algo que devia estar ignorado não está.

POWERSHELL

```
git status
```

LETRA	SIGNIFICA	ONDE APARECE
U	Untracked: arquivo novo que o Git ainda não acompanha.	Lista "Changes"
A	Added: arquivo já preparado (staged) para o próximo commit.	Lista "Staged Changes"
M	Modified: arquivo que já estava no repositório e foi alterado.	Lista "Changes"

20 Faça o primeiro commit

POWERSHELL

```
git add .
git status
git commit -m "Primeiro commit: molde WebView"
```

Ou digite a mensagem no campo do Source Control e clique em **Commit**. Antes de commitar, leia o [git status](#): se aparecer `node_modules`, `keystore.properties`, `jks` ou `.env`, pare — o `.gitignore` não está funcionando. Depois do commit, o painel **Graph**, que dizia “não há itens de histórico”, passa a mostrar o commit.

21 Entenda os avisos de CRLF

No Windows, o Git avisa “LF will be replaced by CRLF the next time Git touches it” em dezenas de arquivos. **Não é erro**: é só o Git informando que vai ajustar o final de linha. O `.gitattributes` do passo 9 já resolve para todo mundo que clonar, e mantém o `gradlew` sempre com LF (com CRLF ele quebra no Linux, no macOS e em CI). Se quiser silenciar os avisos só na sua máquina:

```
git config core.autocrlf true
```

22 Envie para o GitHub

Crie um repositório **vazio e privado** no GitHub (sem README, sem .gitignore — nós já temos os nossos) e conecte. No primeiro push o VS Code abre o navegador para você entrar na conta:

POWERSHELL

```
git remote add origin https://github.com/<org>/<repositorio>.git
git push -u origin main
```

23 Clone em outra máquina

POWERSHELL

```
git clone https://github.com/<org>/<repositorio>.git
cd <repositorio>
npm install
npx cap sync
npx cap open android
```

O clone não traz `node_modules` nem o `local.properties` (o Android Studio o recria sozinho) nem a chave de assinatura, que você recebe do responsável. É exatamente por isso que essas coisas estão no .gitignore.

O que vai e o que não vai para o repositório

VAI PARA O GIT	FICA DE FORA
www/, capacitor.config.json, package.json e package-lock.json	node_modules/
android/ (os arquivos Gradle e o wrapper: são o que o projeto tem de seu)	android/build, android/app/build, android/.gradle, local.properties
resources/, docs/, .gitignore, .gitattributes, README	keystore.properties, *.jks, *.keystore, .env
keystore.properties.example (só com placeholders)	*.aab, *.apk

NOTA

O Capacitor já cria `android/.gitignore` e `android/app/.gitignore` (aparecem no painel como “android” e “android\app”). Eles cuidam do que o `cap sync` gera. O .gitignore da raiz cobre o resto. Os dois trabalham juntos.

Caso real: o git add . antes do .gitignore

Na primeira montagem deste projeto, o `git add .` foi rodado antes de existir .gitignore. O painel mostrou **2473 alterações**, quase todas de `node_modules`. Nada estava perdido: o add só coloca arquivos na área de preparação, e nenhum commit tinha sido feito. A sequência que resolveu:

PASSO	COMANDO	RESULTADO
1	<code>git reset</code>	Tira tudo da staging sem apagar nenhum arquivo. Os arquivos voltam a ser “U”.
2	<code>git rm -r --cached .</code>	Deu fatal: pathspec '.' did not match any files. É esperado: não havia mais nada na staging. O passo 1 já tinha bastado; este só é necessário se o <code>git reset</code> falhar (Git muito antigo).
3	criar o .gitignore	O painel continuou com os mesmos arquivos “U” até o .gitignore existir.
4	<code>git add .</code> de novo	Caiu de 2473 para 62 arquivos, e o <code>node_modules</code> sumiu da lista.

Recuperar de outros erros de Git

SITUAÇÃO	O QUE FAZER
node_modules já entrou em um commit	<code>git rm -r --cached node_modules</code> , depois <code>git commit -m "Remove node_modules"</code>
Um arquivo continua aparecendo mesmo no .gitignore	Ele já estava no índice. <code>git rm --cached <arquivo></code> e commit.
Errei a mensagem ou esqueci um arquivo no último commit	<code>git reset --soft HEAD~1</code> desfaz o commit e mantém tudo staged.
Comitei senha, .jks ou .env	Trate como comprometido: gere outra chave/senha e avise o responsável antes de mexer no histórico.

PARTE 4 — RODAR E DAR ACABAMENTO

Android Studio, emulador, ícone e push

24 Abra o projeto no Android Studio

```
npx cap open android
```

POWERSHELL

O Android Studio abre a pasta **android** (nunca a raiz do projeto). Se ele perguntar sobre confiar no projeto, confirme. Na primeira vez fica alguns minutos “sincronizando o Gradle” — é normal, espere terminar. Conferências:

ONDE	O QUÊ
Settings › Build Tools › Gradle	O Gradle JDK é o 17 ou o 21 que você escolheu no passo 6 (nunca o 25).
Barra de ferramentas	O botão “Sync Project with Gradle Files” (ícone de elefante) refaz a sincronização depois de qualquer mudança nos .gradle.
Avisos de atualização	Se o Studio oferecer o AGP Upgrade Assistant ou pedir para atualizar o Gradle, leia antes de aceitar. Suba o Android Gradle Plugin e o Gradle juntos, nunca só um deles.

25 Crie um emulador ou use o celular

Emulador: ícone de celular no topo (Device Manager) › Create Device › Pixel 7 › uma imagem de sistema Android 14 ou mais nova (de preferência a API 36) › Finish. **Celular:** em Configurações › Sobre o telefone, toque 7 vezes em Número da versão; nas Opções do desenvolvedor, ative a Depuração USB e conecte por cabo — o aparelho aparece na lista.

26 Rode o app

Clique no botão **Run** (triângulo verde). O app instala e abre. **O que você deve ver:** a tela navy com a logo e o spinner por cerca de um segundo e meio, e então a tela de login da 1Norte dentro do app, sem barra de endereço. Se viu isso, o app está funcionando.

ROTINA DIÁRIA

Ciclo de trabalho: editou algo em `www/` › `npx cap sync` › Run › `git add .` e commit. Se algum passo for esquecido, o app roda a versão antiga.

27 Gere o ícone e a splash em todos os tamanhos

O Android pede o ícone em dezenas de tamanhos. Crie a pasta **resources** na raiz com **icon.png** (quadrado, 1024 × 1024 px) e **splash.png** (quadrado, 2732 × 2732 px). Depois rode:

POWERSHELL

```
npm install --save-dev @capacitor/assets
npx @capacitor/assets generate `
  --iconBackgroundColor '#031835' `
  --splashBackgroundColor '#031835'
npx cap sync
```

28 Ligue as notificações push (opcional)

O sistema da 1Norte não dispara push; quem manda é a Sayan, pelo painel do OneSignal. (1) Crie uma conta grátis em onesignal.com e um app do tipo Cordova / Capacitor. (2) Siga o assistente para ligar o Firebase Cloud Messaging, o serviço do Google que entrega as notificações no Android. (3) Copie o **OneSignal App ID** para o campo `ONESIGNAL_APP_ID` do `www/config.js`. (4) Instale o plugin e sincronize:

POWERSHELL

```
npm install onesignal-cordova-plugin
npx cap sync
```

Enquanto o campo estiver "", o app funciona normalmente, só sem push.

PARTE 5 — PUBLICAR

Gerar o pacote e subir na loja

29 Defina a versão

Abra `android/app/build.gradle` e procure o bloco `defaultConfig`. Se o seu arquivo escreve `versionCode = 1` com o sinal de igual, mantenha o estilo dele:

android/app/build.gradle

```
defaultConfig {
    applicationId "tech.nortel.sayan.beneficios"
    versionCode 1 // inteiro. Sobe a cada envio, nunca repete
    versionName "1.0" // o texto que o usuário vê na loja
}
```

O `versionCode` só sobe e não aceita ponto: escrever 2.5 ali não funciona. A versão “comercial”, com ponto, vai no `versionName`.

30 Crie a chave de assinatura

Todo app Android é assinado por uma chave que prova que a atualização veio de quem publicou o original. Na Play Store, com o **Play App Signing** (padrão em apps novos), a chave que você cria é a de **upload**; por isso o alias termina em “-upload”. **Se ela for perdida, você depende do suporte do Google para trocá-la**, e o `applicationId` não pode ser reaproveitado. Para o app da Sayan, peça a chave e as senhas ao responsável. Para um app **novo**, crie a chave com o `keytool`, que vem dentro do JDK (use o terminal do passo 6):

POWERSHELL

```
keytool -genkeypair -v -keystore C:/chaves/cliente-upload.jks -alias cliente-upload `
  -keyalg RSA -keysize 2048 -validity 10000
```

Guarde o arquivo `.jks` e as senhas em lugar seguro, **fora** da pasta do projeto e fora do Git.

31 Ligue o Gradle ao keystore.properties

Crie `android/keystore.properties`. Use barra normal `/`, nunca a invertida: em arquivos `.properties` ela é caractere de escape, e o build falha com “Keystore file not found” apontando um caminho torto que você vai jurar que está certo.

android/keystore.properties

```
storeFile=C:/caminho/para/a/chave/sayan-upload
storePassword=<peça ao responsável>
keyAlias=sayan-upload
keyPassword=<peça ao responsável>
```

Depois, no **android/app/build.gradle**, faça o Gradle ler esse arquivo (se o seu ainda não fizer). O trecho abaixo vai antes do bloco **android {**, e o **signingConfigs** e o **buildTypes** ficam dentro dele:

android/app/build.gradle

```
def keystorePropertiesFile = rootProject.file("keystore.properties")
def keystoreProperties = new Properties()
if (keystorePropertiesFile.exists()) {
    keystoreProperties.load(new FileInputStream(keystorePropertiesFile))
}

android {
    signingConfigs {
        release {
            if (keystorePropertiesFile.exists()) {
                storeFile file(keystoreProperties['storeFile'])
                storePassword keystoreProperties['storePassword']
                keyAlias keystoreProperties['keyAlias']
                keyPassword keystoreProperties['keyPassword']
            }
        }
    }
    buildTypes {
        release {
            signingConfig signingConfigs.release
        }
    }
}
```

IMPORTANTE

Confirme que o **.gitignore** tem as linhas **keystore.properties**, ***.jks** e ***.keystore** (passo 9) e rode **git status** antes de todo commit. Chaves e senhas nunca entram no Git.

32 Gere o pacote assinado

No terminal, entre na pasta **android**, aponte para o JDK do passo 6 (o **\$env:JAVA_HOME** vale só nesta janela) e gere o bundle:

POWERSHELL

```
cd android
$env:JAVA_HOME = "C:\Users\<seu-usuario>\.jdk\jbr-21.0.x"
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
.\gradlew --version # confira a linha "JVM:"
.\gradlew bundleRelease
```

Ao terminar, o arquivo está em **android/app/build/outputs/bundle/release/app-release.aab**. **Não use** o assistente do Android Studio (Build > Generate Signed Bundle): a assinatura já está no Gradle, e o assistente salva o arquivo em outro lugar — foi o que nos fez procurar um **.aab** que não estava onde devia.

33 Suba no Play Console

Este é o caminho do **teste fechado**, a faixa em que o app da Sayan está hoje.

1. Entre no Play Console e selecione o app.
2. Menu lateral: **Testar e lançar** > **Teste** > **Teste fechado**.
3. Clique em **Criar nova versão**, no canto superior direito.

4. Arraste o .aab para a área de upload. Se o bundle já foi enviado antes, use **Adicionar da biblioteca** e escolha o versionCode — evita recompilar.
5. Preencha as notas da versão entre as tags do idioma: `<pt-BR> 0 que mudou... </pt-BR>`
6. **Avançar**, confira os avisos e **Salvar**. Isso ainda não envia nada ao Google.
7. Em **Visão geral da publicação**, confira a lista e clique em **Enviar mudanças para revisão**. É aqui que o envio acontece.
8. Nas abas **Países / regiões** e **Testadores**, adicione o Brasil e a lista de e-mails (contas Google).

ATENÇÃO

Os links de convite só aparecem depois que o Google aprovar a versão. Até lá a aba de testadores mostra “os links serão exibidos aqui quando você publicar o app”, e ninguém consegue aceitar. A revisão costuma levar até 7 dias.

PARTE 6 — O MOTIVO DE TUDO ISSO

Reaproveitar para o próximo app

Com o molde de pé, um app novo leva algumas horas, não semanas. Parta do repositório, tire o que é do cliente anterior e mexa só no que está na tabela.

POWERSHELL

```
git clone https://github.com/<org>/<molde>.git app-novo-cliente
cd app-novo-cliente
Remove-Item -Recurse -Force .git, android # tira o histórico e o android antigo
git init
npm install
npx cap add android
npx cap sync
```

O QUE TROCAR	ONDE	CUIDADO
Endereço do sistema	<code>www/config.js</code> · <code>SYSTEM_URL</code>	Tem que ser a página de login, não a home.
Domínios permitidos	<code>capacitor.config.json</code> · <code>allowNavigation</code>	Só os do sistema. Quanto menor a lista, menos o app declara na loja.
Identificador do app	<code>capacitor.config.json</code> · <code>appld</code>	Único por app e permanente. Use o domínio do cliente ao contrário: <code>br.com.cliente.app</code> .
Nome do app	<code>capacitor.config.json</code> · <code>appName</code>	Até 30 caracteres na Play Store.
Cores da marca	<code>www/styles.css</code> · <code>--brand</code> e <code>--accent</code> ; <code>capacitor.config.json</code> · <code>backgroundColor</code>	A cor da splash aparece em dois lugares. Trocar só um causa um “pisca” feio na abertura.
Logo, ícone e splash	<code>www/logo.png</code> ; <code>resources/icon.png</code> e <code>splash.png</code>	Rode <code>npx @capacitor/assets generate</code> de novo depois de trocar.
App do OneSignal	<code>www/config.js</code> · <code>ONESIGNAL_APP_ID</code>	Um app do OneSignal por cliente. Reaproveitar manda notificação para os usuários errados.
Chave de assinatura	<code>android/keystore.properties</code>	Uma chave nova para cada app. Nunca reutilize a de outro cliente.

IMPORTANTE

Antes de qualquer linha de código de um app novo, confira o **tipo da conta de desenvolvedor**. Desde 31/08/2024 o Google exige conta de **organização** (empresa, com número D-U-N-S) para publicar quatro categorias: serviços financeiros, apps de saúde, apps com VPN e apps governamentais. Conta pessoal não publica nenhuma delas. Foi exatamente assim que o app da Sayan foi rejeitado — depois de toda a ficha preenchida e o bundle enviado. Como a maioria dos nossos clientes é da área de saúde, isso vale para quase todo app novo: checar a conta é o **passo zero**.

Um roteiro de três horas para o próximo app

HORA	O QUE FAZER
0h	Confirmar o tipo de conta na Play Store e criar o app no Console (o appld já fica reservado).
0h30	Copiar o molde, trocar os oito itens da tabela acima, rodar <code>npx cap sync</code> .
1h	<code>npx cap add android</code> , rodar no emulador, conferir o login do cliente funcionando dentro do app.
1h30	Gerar a chave nova, guardar em lugar seguro, configurar o <code>keystore.properties</code> , gerar o <code>.aab</code> (com o JDK certo).
2h	Preparar os materiais da loja: ícone 512×512, gráfico 1024×500, quatro capturas em 1080×1920.
2h30	Preencher a ficha do app no Console e enviar para revisão.

O que costuma estourar esse prazo não é o código: é a ficha do app, a política de privacidade num endereço público e as capturas de tela no formato certo. Deixe esses três itens pedidos ao cliente antes de começar.

O que vale transformar em base reutilizável

Depois do segundo ou terceiro app compensa manter um repositório-molde com tudo pronto e só os pontos de troca marcados. Duas coisas que valem entrar nele desde já: um **README** com a tabela acima e as versões de Android Studio, JDK, Gradle e Capacitor que funcionaram; e um `keystore.properties.example`, para ninguém inventar o formato do arquivo na pressa e perder meia hora com barra invertida:

```
storeFile=C:/caminho/para/a/chave/cliente-upload
storePassword=SUBSTITUA
keyAlias=cliente-upload
keyPassword=SUBSTITUA
```

`android/keystore.properties.example`

REFERÊNCIA

Quando der errado

MENSAGEM	CAUSA	SOLUÇÃO
Unsupported class file major version 69	O Gradle está usando o JDK 25 (JAVA_HOME ou Gradle JDK).	Use o 17 ou o 21 (passo 6). Confirme com <code>./gradlew --version</code> . O caminho do Studio está em <code>android/.gradle/config.properties</code> .
invalid source release: 21	O projeto pede Java 21 e o JDK em uso é mais antigo.	Instale o JDK 21 e aponte o Gradle JDK e o JAVA_HOME para ele.
ERROR: JAVA_HOME is not set	A variável não foi definida nesta janela.	Rode a linha <code>\$env:JAVA_HOME = ...</code> antes do gradlew, na mesma janela.
Keystore file not found	Barra invertida no <code>keystore.properties</code> .	Troque \ por / e confirme que o arquivo existe naquele caminho.
Get Key failed: extra data at the end	Senha da chave incorreta.	Confira <code>keyPassword</code> e <code>storePassword</code> .

MENSAGEM	CAUSA	SOLUÇÃO
"O código de versão N já foi usado"	versionCode repetido.	Aumente o número e gere o bundle de novo.
"Assinado com uma chave incorreta"	O build usou outra keystore.	Compare o SHA-1 de <code>keytool -list -v -keystore <arquivo></code> com o do Play Console.
"O nível da API precisa ser pelo menos 36"	targetSdk desatualizado.	Subir só o targetSdk não basta: o Android Gradle Plugin precisa saber compilar contra aquela API. Suba AGP e Gradle juntos.
Editei o <code>www/</code> e nada mudou	Faltou sincronizar.	<code>npx cap sync</code> e rode de novo.
Tela branca no app	Erro de JavaScript no <code>app.js</code> .	No Chrome do computador, abra <code>chrome://inspect</code> com o celular conectado: dá para ver o console do app.
'git' não é reconhecido	Git não instalado, ou terminal antigo.	Instale o Git (passo 2) e abra um terminal novo.
Author identity unknown	O Git não sabe quem você é.	<code>git config --global user.name</code> e <code>user.email</code> (passo 2).
Painel com milhares de alterações (2473)	<code>node_modules</code> sem <code>.gitignore</code> .	Passo 9; se já rodou o add, faça <code>git reset</code> e crie o <code>.gitignore</code> .
<code>node_modules</code> aparece mesmo com <code>.gitignore</code>	Já estava no índice do Git.	<code>git rm -r --cached node_modules</code> e commit.

NÃO NEGOCIÁVEL

Sete regras

- 1 Nunca commitar `keystore.properties`, `.jks`, `.keystore` ou `.env`. O `.gitignore` vem antes do primeiro `git add ..`
- 2 Nunca mudar o `applicationId` de um app publicado. É permanente e não pode ser reaproveitado — nem por outro app, nem depois de apagado.
- 3 Cada app novo tem chave e `applicationId` próprios. Reaproveitar a chave de um cliente em outro app mistura duas coisas que precisam viver separadas.
- 4 O `versionCode` só sobe. Nunca repita, nunca diminua.
- 5 Nunca gerar o pacote com o **JDK 25**. Confirme o JDK com `.\gradlew --version` antes de todo `bundleRelease`.
- 6 Não adicione domínios ao `allowNavigation` sem discutir antes — isso muda o que o app precisa declarar na Play Store.
- 7 Antes de publicar uma captura de tela, confira se não há CPF, número de cartão ou dado real de cliente visível. Já aconteceu.

Erro que este material não cobre: fale com o responsável pelo projeto antes de tentar contornar. Em build e publicação de app, a maioria dos atalhos custa mais caro que a pergunta.